

Technical Specification: Multi-Tenant Lead Response Bridge

Architectural Blueprint: Webhook Integration → OAuth Identity → SendGrid Delivery

1. System Overview

The goal is to build a high-performance middleware that listens for Luxury Presence (LP) webhooks and executes authenticated lead responses via SendGrid, using the specific client's identity verified through Google OAuth. This ensures "instant speed" delivery with "human-level" authenticity.

2. Infrastructure Components

A. The Ingestion Layer (Webhooks)

Each client is assigned a unique UUID. The webhook listener must handle POST requests at: `/v1/webhooks/incoming/{client_uuid}`.

- **Validation:** The system must first verify the `client_uuid` exists in the database.
- **Normalization:** Map varying LP payload fields (e.g., `lead_email`, `property_ref`) to an internal Lead Object.

B. The Identity Store (Database Schema)

The `Clients` table must store the following encrypted fields:

```
{
  "client_id": "UUID",
  "client_email": "agent@domain.com",
  "gmail_oauth": {
    "access_token": "STRING",
    "refresh_token": "STRING",
    "expiry": "TIMESTAMP"
  },
  "sendgrid_api_key": "STRING",
  "template_config": {
    "subject": "Question about {{property_address}}",
    "body": "Hi {{first_name}}, I saw your interest in {{property_address}}..."
  }
}
```

```
}
```

3. The Execution Logic (The "Bridge")

When a payload is received, the developer must implement the following sequential logic:

Step 1: Identity Resolution

Fetch the client record. If the OAuth `access_token` is expired (current time > expiry), execute the **Token Refresh Flow** using the `refresh_token` via the Google OAuth2 endpoint (`https://oauth2.googleapis.com/token`). This ensures the "From" address remains valid.

Step 2: Payload Personalization

Apply lead data to the client's specific template. Use a simple placeholder replacement logic:

```
function generateEmail(template, leadData) {  
    return template.replace(/{{(.*)}}/g, (match, key) => leadData[key.trim()] || "");  
}
```

Step 3: Dispatch via SendGrid API

The API call to SendGrid (`v3/mail/send`) must use the **Agent's specific API Key** or a Master Key with ****Sub-user impersonation****. Crucially, the `from` field must match the agent's OAuth-verified email.

Critical Header Requirement: Set the `Reply-To` header specifically to the agent's email. This ensures that when the lead hits "Reply," the conversation moves directly into the agent's Gmail inbox, bypassing the system.

4. Technical Workflow Diagram

Phase	Technology	Requirement
Ingestion	Node.js / Express	Asynchronous processing (return 200 OK immediately).
Auth Check	Google OAuth 2.0	Automatic refresh if $T_{\{now\}} > T_{\{expiry\}} - 300s$.

Dispatch	SendGrid v3 API	Plain text format to maximize "human" feel.
----------	-----------------	---

Logging	PostgreSQL / Redis	Track <code>message_id</code> for delivery verification.
---------	--------------------	--

5. Scaling & Security Considerations

Token Security

Refresh tokens must be encrypted at rest (AES-256). Never return tokens in the API response. The middleware should use a "Service Account" logic to handle the backend-to-backend communication between Google and SendGrid.

Error Handling & Fallback

If the Google OAuth identity is revoked by the user, the system should catch the 401 error. The developer must implement a **Safe Fallback** where the email is still sent via a generic system domain, but an alert is triggered to the admin to re-authenticate the client.

Deliverability Optimization

To avoid spam filters, the system must NOT use heavy HTML/CSS. The core strategy is **"Plain Text with Personalization."** This mimics the behavior of an agent typing a manual email from their smartphone.

6. Getting Started (Developer Checklist)

1. Set up a Google Cloud Console Project with Gmail API enabled.
2. Configure the OAuth Consent Screen for "External" use.
3. Implement the `/auth/callback` route to capture tokens.
4. Build the `/webhooks/incoming/:id` route.
5. Test the SendGrid integration using `curl` to ensure the `From` name overrides are working.